P.Vishwa Nandan(192473015)

TASK 1 – GUI DESIGN REVIEW (6 Marks)

1. Aim

To analyse and correct a poorly designed student fee-payment application by separating the graphical user interface from the business logic, improving event handling, validating user input, and handling database operations safely.

2. Problem Description

A junior developer created a student fee-payment window with three buttons:

- Pay
- Clear
- Receipt

All three buttons use the same event listener. The complete payment logic, database operations, and input handling are written inside the actionPerformed() method.

This creates several design problems, including poor separation of responsibilities, incorrect event handling, unsafe SQL queries, and an unresponsive application.

The application must be redesigned using a separate FeeService class to perform validation and payment operations.

3. Task 1.1 – Diagnose the Design (2 Marks)

3.1 Design Deficiencies

The original program contains the following design problems.

1. No Separation Between Interface and Business Logic

The complete payment logic is written inside the actionPerformed() method.

Problem: The interface and payment operations are tightly connected. Changing the payment process requires modifying the GUI code.

Solution: Create a separate FeeService class to handle validation and payment operations.

2. Incorrect String Comparison

The code uses:

if (e.getActionCommand() == "Pay")

The == operator compares object references rather than string contents.

Problem: The program may fail to identify the correct button action.

Solution: Use:

if ("Pay".equals(e.getActionCommand()))

3. No Proper Input Validation

The code directly converts the amount using:

double a = Double.parseDouble(amount.getText());

It does not check whether the input is empty, invalid, zero, or negative.

Problem: Invalid input may cause an exception or an incorrect payment amount.

Solution: Validate the registration number and amount before processing payment.

4. Database Operations on the Event-Dispatch Thread

The database connection and SQL query are executed inside the button event handler.

Problem: Database operations may take time, causing the application window to freeze and preventing the user from interacting with it.

Solution: Perform database operations in a background thread.

5. Use of Thread.sleep()

The program contains:

Thread.sleep(3000);

Problem: It pauses the event-handling thread for three seconds, making the window unresponsive.

Solution: Perform long-running operations in a background thread. In a real application, bank confirmation should be handled through a payment gateway instead of using Thread.sleep().

6. No Proper Feedback to the User

The program displays:

System.out.println("Paid");

Problem: The payment message appears only in the console, not in the application window.

Solution: Display payment success or failure messages using a label or dialog box.

7. Receipt Button Is Not Implemented

The original code uses else for all actions other than Pay.

Problem: Clicking the Receipt button clears the fields instead of displaying a payment receipt.

Solution: Implement separate event handling for the Pay, Clear, and Receipt buttons.

8. Unsafe SQL Statement

The program directly joins user input into the SQL query:

"INSERT INTO fees VALUES('" + regNo.getText() + "'," + a + ")"

Problem: This can allow SQL injection and may cause SQL errors when user input contains special characters.

Solution: Use PreparedStatement with parameterized queries.

9. Database Connection Is Not Closed

The program creates a database connection but does not close it.

Problem: Unclosed database connections can consume resources and eventually cause connection exhaustion.

Solution: Use try-with-resources to close database connections and statements automatically.

10. No Window Closing Handling

The original program does not define what happens when the user clicks the window close button.

Problem: The application may not close or release its resources properly.

Solution: Use WindowAdapter to handle the window-closing event.

11. Checked Exceptions Are Not Handled

The code performs database operations and uses Thread.sleep() without handling their checked exceptions.

Problem: The program will not compile as written because exceptions such as SQLException and InterruptedException must be handled or declared.

Solution: Use appropriate try-catch blocks and restore the thread's interrupted status when necessary.

3.2 Summary of Design Deficiencies

No.	Deficiency	Possible Problem
1	No separation of logic	Difficult to maintain
2	Uses == for strings	Incorrect button detection
3	No input validation	Invalid payment values
4	Database work on event thread	Window freezes
5	Uses Thread.sleep()	Application becomes unresponsive

6	Uses console feedback	User cannot see payment status
7	Receipt button not implemented	Incorrect button behaviour
8	SQL string concatenation	SQL injection risk
9	Connection not closed	Resource leakage
10	No window closing handler	Improper application closing
11	Unhandled checked exceptions	Compilation and runtime problems

4. Task 1.2 – Repair the Design (3 Marks)

4.1 Proposed Design

The application is redesigned using two main classes.

1. FeeWindow

Responsibilities:

- Display the payment window.
- Collect the registration number.
- Collect the payment amount.
- Handle Pay, Clear, and Receipt button actions.
- Display payment success and error messages.

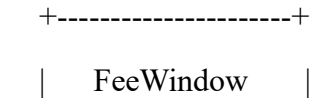
2. FeeService

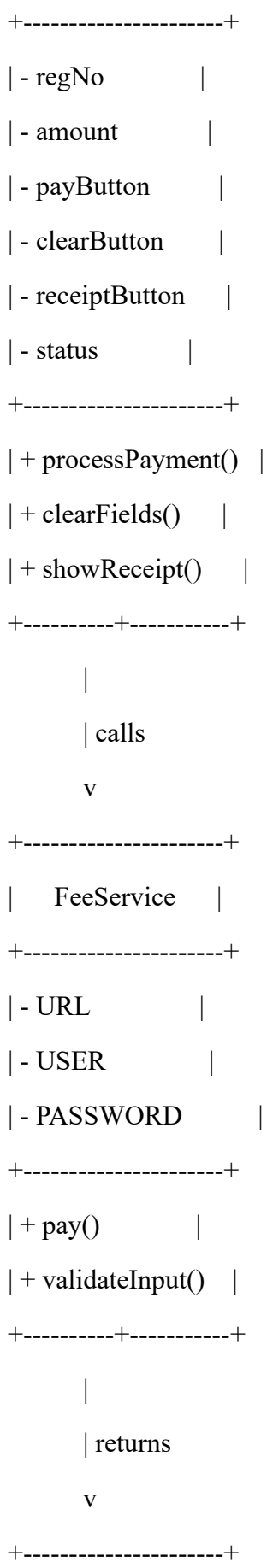
Responsibilities:

- Validate the registration number.
- Validate the payment amount.
- Connect to the MySQL database.
- Perform the payment-record insertion.
- Return the payment receipt.
- Handle database-related errors.

A PaymentReceipt class stores the details of a successful payment.

4.2 Class Diagram




- regNo
- amount
- payButton
- clearButton
- receiptButton
- status
+ processPayment()
+ clearFields()
+ showReceipt()
calls
FeeService
- URL
- USER
- PASSWORD
+ pay()
+ validateInput()
returns

| PaymentReceipt |

+----------------------+

| - paymentId |

| - regNo |

| - amount |

+----------------------+

| + toString() |

+----------------------+

4.3 Event Handling Design

Each button has its own event listener.

Button	**Listener**	**Action**
Pay	ActionListener	Starts payment processing
Clear	ActionListener	Clears input fields
Receipt	ActionListener	Displays the last successful receipt
Window Close	WindowAdapter	Handles application closing

4.4 Database Setup

Create the database and table in MySQL Workbench.

```
CREATE DATABASE college_fees;


USE college_fees;


CREATE TABLE fee_payments (
    payment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    reg_no VARCHAR(30) NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    paid_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4.5 Java Program 1 – FeeService.java

Explanation:

1. The pay() method receives the registration number and amount.
2. It checks whether the registration number is empty.
3. It verifies whether the entered amount is a valid positive number.
4. It simulates bank confirmation for three seconds.
5. It connects to the MySQL database.
6. It uses PreparedStatement to insert the payment safely.
7. It returns a PaymentReceipt object after successful insertion.
8. It handles database errors using a custom exception.

The three-second delay is only for demonstration. A real application should integrate with an actual payment gateway rather than treating a delay as bank confirmation.

4.6 Java Program 2 – FeeWindow.java

Explanation:

1. The window contains registration number and payment amount fields.
2. The Pay button starts the payment operation.
3. The Clear button clears the input fields.
4. The Receipt button displays the last successful payment details.
5. The payment operation runs in a background thread.
6. EventQueue.invokeLater() updates the interface safely.
7. WindowAdapter handles the window-closing event.
8. The window remains responsive while the payment is being processed.

4.7 Expected Output

Window Before Payment:

Student Fee Payment

Registration Number: STU101

Payment Amount: 5000

[Pay] [Clear]

[Receipt]

Enter payment details.

During Payment:

Processing payment...

After Successful Payment:

Payment successful.

After Clicking Receipt:

Payment ID: 1 | Registration No: STU101 | Amount: Rs. 5000.00

For Invalid Amount:

Please enter a valid payment amount.

The payment ID and successful database insertion depend on the MySQL database configuration.

4.8 Improvements in the Redesigned Program

Original Design	Redesigned Design
All logic in one listener	Separate FeeWindow and FeeService
Uses == for strings	Uses separate button listeners
No validation	Validates registration number and amount
Database work on event thread	Background thread performs payment
Uses console output	Displays status in the window
Receipt button clears fields	Receipt button displays payment details
Unsafe SQL query	Uses PreparedStatement

Connection not closed	Uses try-with-resources
No window event handling	Uses WindowAdapter
Unhandled exceptions	Uses exception handling

5. Task 1.3 – Design Defence (1 Mark)

Why Is the Redesigned Program Easier to Test and Maintain?

The redesigned program separates the GUI from the payment logic. The FeeWindow class handles user interaction, while the FeeService class performs validation and database operations. This makes each class easier to test independently. Changes to payment rules can be made without modifying the interface. Separate event listeners also improve readability and make the application easier to maintain and extend.

6. Alternative Concept and Complexity Analysis

6.1 Separate Listener Objects

Instead of using one listener with an if-else chain, each button can have its own listener object.

For example:

```
pay.addActionListener(e -> processPayment());


clear.addActionListener(e -> clearFields());


receipt.addActionListener(e -> showReceipt());
```

Each listener directly performs its assigned operation.

This avoids checking the action command against multiple button names.

6.2 Comparison of Both Approaches

Approach 1 – Single Listener with if-else

```
public void actionPerformed(ActionEvent e) {


   if ("Pay".equals(e.getActionCommand())) {


      processPayment();
```

```
    } else if ("Clear".equals(e.getActionCommand())) {

        clearFields();

    } else if ("Receipt".equals(e.getActionCommand())) {

        showReceipt();
    }
}
```

In this approach, the program checks the action command to identify which button was clicked.

Approach 2 – Separate Listener Objects

```
pay.addActionListener(e -> processPayment());

clear.addActionListener(e -> clearFields());

receipt.addActionListener(e -> showReceipt());
```

Each button has its own event-handling code.

6.3 Time Complexity

Let k represent the total number of buttons.

Single Listener:

The program may perform up to k comparisons to identify the button action.

Time complexity:

O(k)

Separate Listener Objects:

The corresponding listener is invoked directly for the button event, without checking an if-else chain.

Time complexity:

O(1) per event-handler dispatch.

6.4 Space Complexity

Single Listener:

Only one shared listener object is required.

Additional listener space: O(1).

Separate Listener Objects:

One listener object is created for each button.

Additional listener space: O(k).

6.5 Comparison Table

Feature	Single Listener	Separate Listeners
Listener objects	One shared object	One per button
Event handling	if-else chain	Direct action
Dispatch time	O(k) comparisons	O(1) handler dispatch
Listener space	O(1)	O(k)
Code readability	Decreases with more buttons	Easier to understand
Maintainability	More difficult for large applications	Easier to modify

6.6 Conclusion of Alternative Approach

For a small application, one shared listener is acceptable. However, separate listeners are easier to understand and maintain when the number of buttons increases.

For larger applications, Swing Action objects can also be used to share the same action between buttons, menu items, and keyboard shortcuts.

7. Final Result

The original student fee-payment application was analysed, and its design deficiencies were identified.

The application was redesigned by separating the graphical user interface and payment operations into different classes. Input validation, PreparedStatement, exception handling, background processing, receipt generation, and window-event handling were implemented.

The alternative event-handling approaches were also compared using time and space complexity.

Thus, the redesigned application provides better separation of responsibilities, improved responsiveness, security, and maintainability.